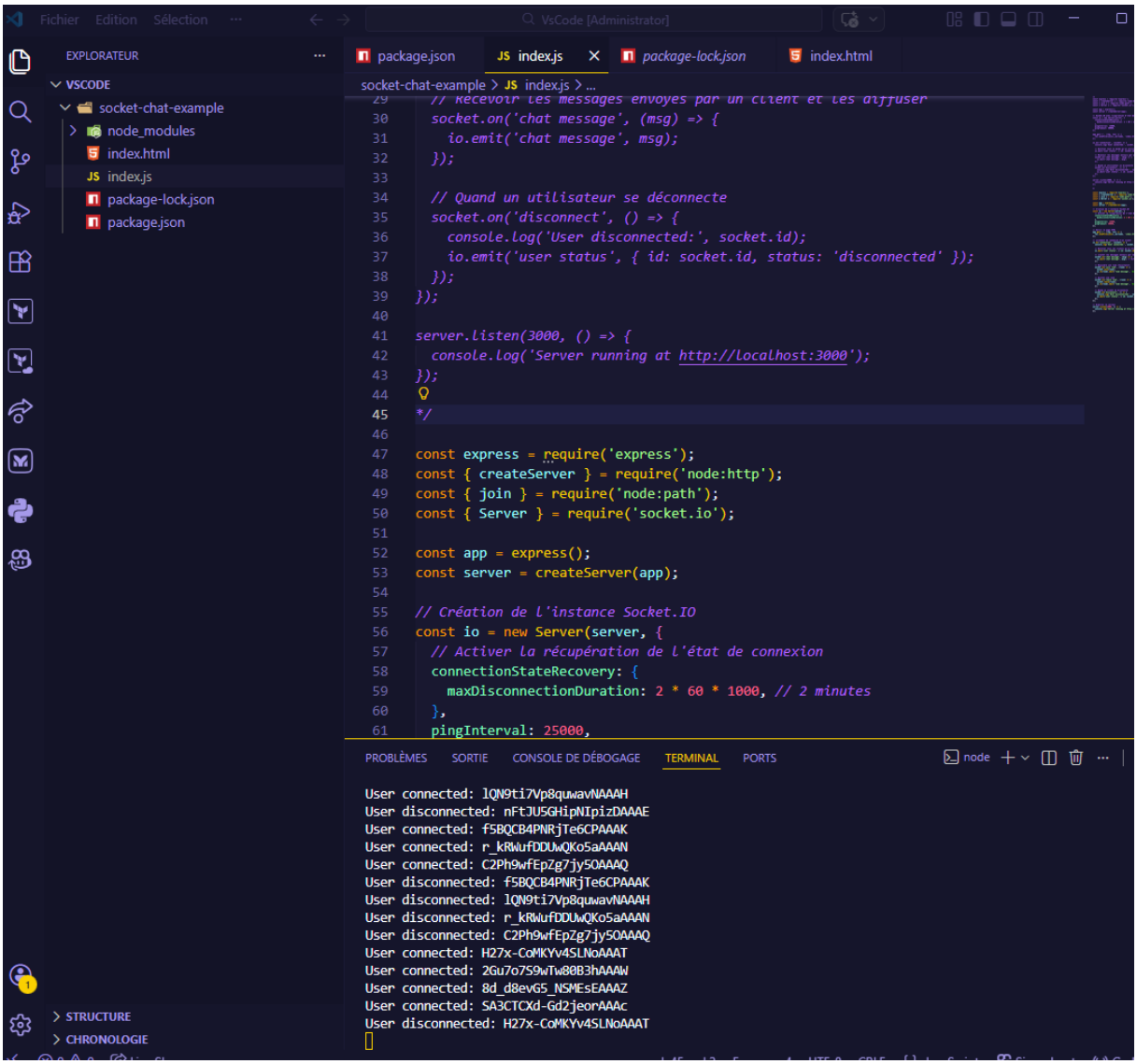
RENDU TP - Guillemette

Les contraintes techniques avérées :

Créer un serveur websocket

— Faire le tutoriel



Fichier Index.js

```
const express = require('express');
const { createServer } = require('node:http');
const { join } = require('node:path');
const { Server } = require('socket.io');

const app = express();
const server = createServer(app);
```

```
// Création de l'instance Socket.IO
const io = new Server(server, {
  // Activer la récupération de l'état de connexion
  connectionStateRecovery: {
    maxDisconnectionDuration: 2 * 60 * 1000, // 2 minutes
  },
  pingInterval: 25000,
  pingTimeout: 60000,
});

// Servir la page HTML
app.get('/', (req, res) => {
  res.sendFile(join(__dirname, 'index.html'));
});

// Événement de connexion d'un client
io.on('connection', (socket) => {
  console.log('User connected:', socket.id);

  // Notifier tous les clients qu'un utilisateur s'est connecté
  io.emit('user status', { id: socket.id, status: 'connected' });

  // Écouter les messages envoyés par les clients
  socket.on('chat message', (msg) => {
    io.emit('chat message', msg); // Diffusion à tous
  });

  // Rejoindre une room (facultatif)
  socket.on('join room', (room) => {
    socket.join(room);
    io.to(room).emit('room message', `User ${socket.id} joined room ${room}`);
  });

  // Quitter une room
  socket.on('leave room', (room) => {
    socket.leave(room);
    io.to(room).emit('room message', `User ${socket.id} left room ${room}`);
  });

  // Quand un client se déconnecte
  socket.on('disconnect', () => {
    console.log('User disconnected:', socket.id);
    io.emit('user status', { id: socket.id, status: 'disconnected' });
  });
});

// Démarrer le serveur
server.listen(3000, () => {
  console.log('Server running at http://localhost:3000');
```

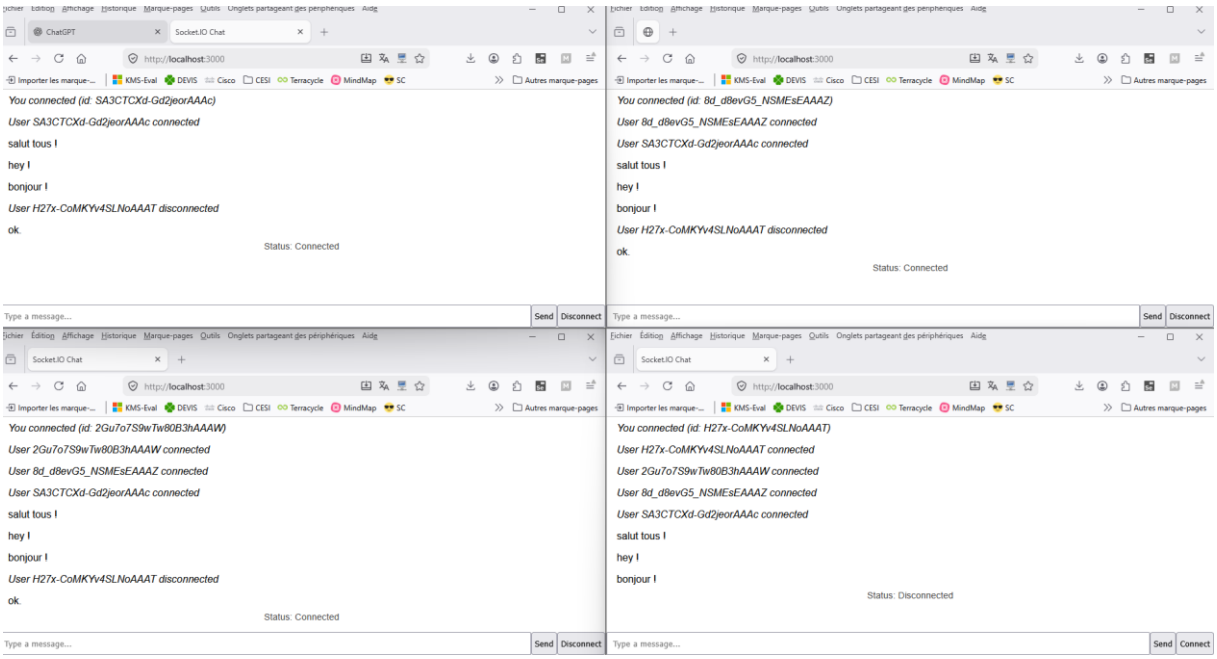
```
});
```

Fichier : Index.html

```
<!DOCTYPE html>
<html>
  <head>
    <meta name="viewport" content="width=device-width,initial-scale=1.0">
    <title>Socket.IO Chat</title>
    <style>
      body { margin: 0; padding-bottom: 3rem; font-family: Arial, sans-serif; }
      #form { position: fixed; bottom: 0; left: 0; right: 0; display: flex; }
      #input { flex: 1; padding: 0.5rem; }
      #messages { list-style-type: none; margin: 0; padding: 0; }
      #messages li { padding: 0.5rem 1rem; }
      #status { text-align: center; font-size: 0.9rem; color: #444; }
    </style>
  </head>
  <body>
    <ul id="messages"></ul>
    <div id="status">Status: Connecting...</div>
    <!-- Formulaire d'envoi -->
    <form id="form">
      <input id="input" autocomplete="off" placeholder="Type a message..." />
      <button>Send</button>
      <button id="toggle-btn">Disconnect</button>
    </form>
    <!-- Charger Socket.IO -->
    <script src="/socket.io/socket.io.js"></script>
    <script>
      // Connexion Socket.IO avec reconnexion automatique
      const socket = io({ autoConnect: true, reconnection: true });

      const form = document.getElementById('form');
      const input = document.getElementById('input');
      const messages = document.getElementById('messages');
      const statusDiv = document.getElementById('status');
      const toggleBtn = document.getElementById('toggle-btn');

      // Ajouter un message dans la liste
      function addMessage(text, italic = false) {
        const item = document.createElement('li');
        if (italic) item.style.fontStyle = 'italic';
        item.textContent = text;
        messages.appendChild(item);
        window.scrollTo(0, document.body.scrollHeight);
      }
```

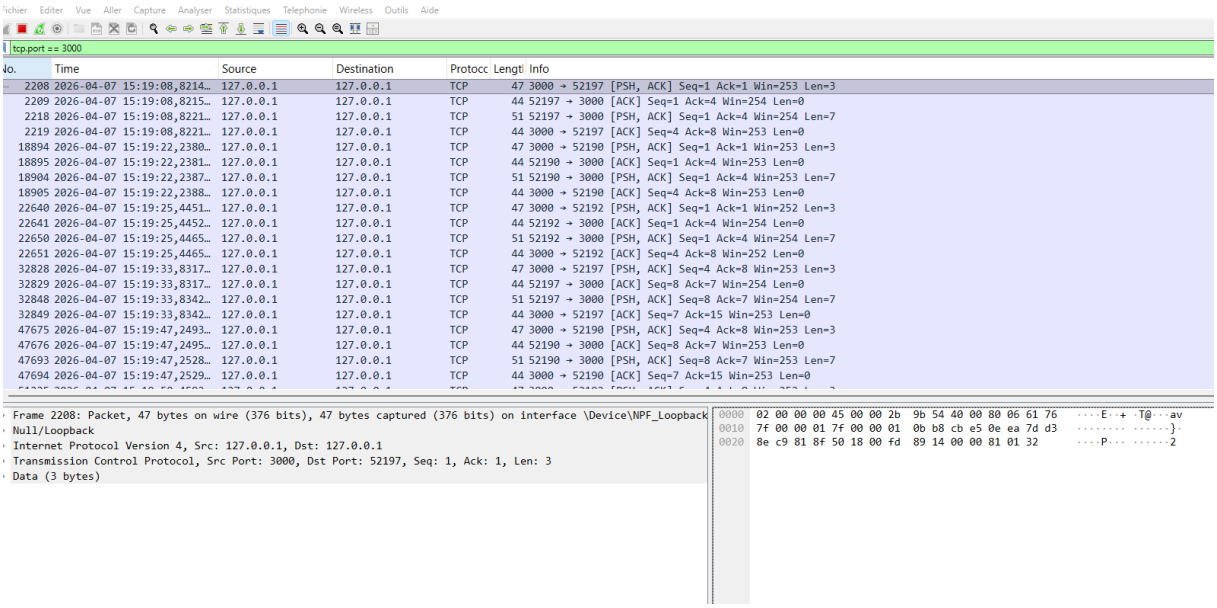
```
      // Envoi du message au serveur
      form.addEventListener('submit', (e) => {
        e.preventDefault();
        if (input.value && socket.connected) {
          socket.emit('chat message', input.value);
          input.value = '';
        }
      });

      // État de connexion affiché
      socket.on('connect', () => {
        statusDiv.textContent = 'Status: Connected';
        toggleBtn.innerText = 'Disconnect';
        addMessage(`You connected (id: ${socket.id})`, true);
      });

      socket.on('disconnect', () => {
        statusDiv.textContent = 'Status: Disconnected';
        toggleBtn.innerText = 'Connect';
      });

      // Réception d'un message normal
      socket.on('chat message', (msg) => {
        addMessage(msg);
      });

      // Réception d'un message de statut utilisateur
      socket.on('user status', (data) => {
        addMessage(`User ${data.id} ${data.status}`, true);
      });

      // Rejoindre ou quitter des rooms (facultatif)
      // Exemple : socket.emit('join room', 'room1');
      socket.on('room message', (msg) => {
        addMessage(msg, true);
      });

      // Bouton Connect / Disconnect
      toggleBtn.addEventListener('click', (e) => {
        e.preventDefault();
        if (socket.connected) socket.disconnect();
        else socket.connect();
      });
    </script>
  </body>
</html>
```

Fonctionne (fait avec tuto) :



— Observer la trace réseau Wireshark/ T-Shark

Connexion Loopback adaptator car sur wifi pas de trafic web socket ou tcp port = 3000 (car on a ouvert notre web socket sur le port 3000)



— *Utiliser Burp Suite*

Créer un programme qui permette de vérifier quels sont les ports réseaux ouverts sur une machine donnée

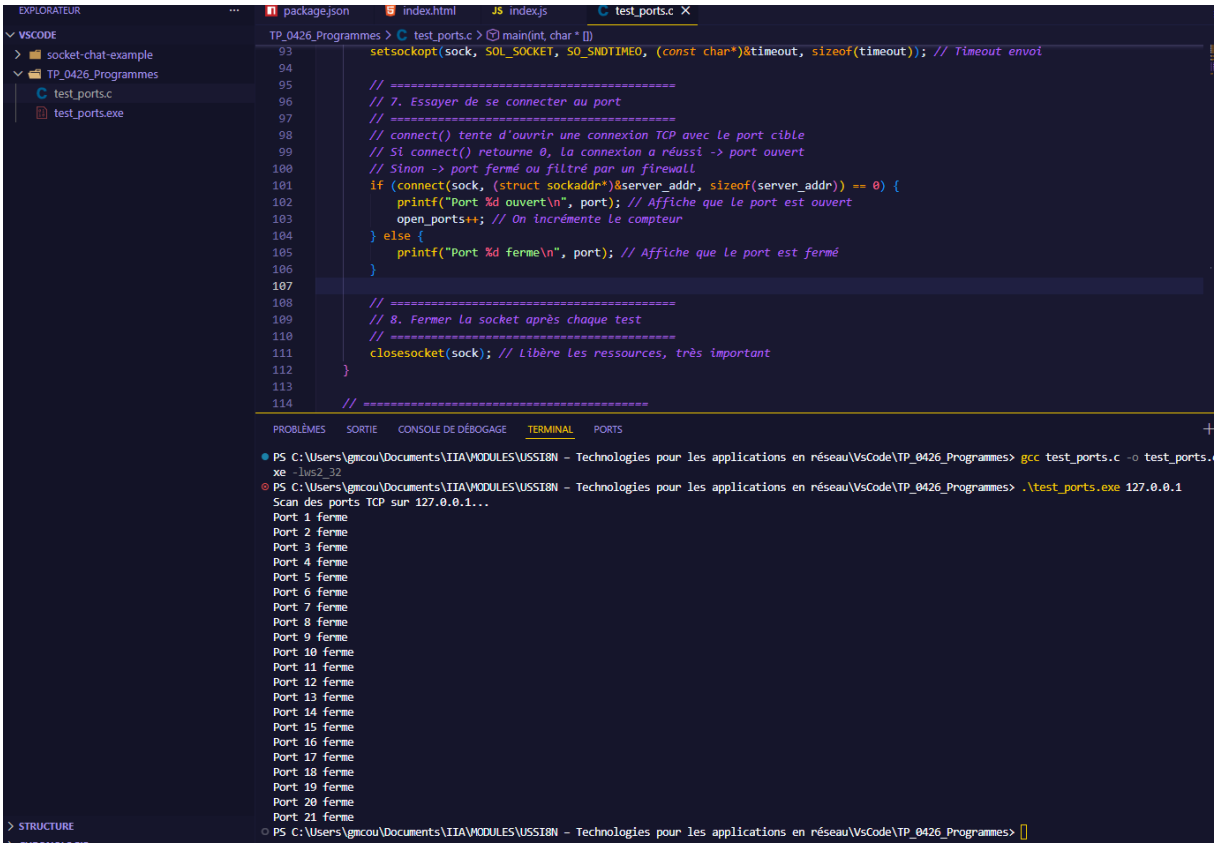
— *Faire cela dans un langage de bas niveau (C, C++, Go, Rust, Java..) :*

Socket en C :

```
#include <stdio.h>      // Pour afficher du texte à l'écran avec printf() et gérer les erreurs avec perror()
#include <stdlib.h>     // Pour quitter le programme avec exit() et convertir des chaînes en nombres avec atoi()
#include <string.h>     // Pour manipuler les chaînes de caractères et mettre à zéro des zones mémoire avec memset()
#include <winsock2.h>   // Pour utiliser les sockets réseau sur Windows (TCP/UDP)
#include <ws2tcpip.h>   // Pour des fonctions réseau supplémentaires (comme inet_pton pour convertir IP)

#pragma comment(lib, "ws2_32.lib")  // Indique au compilateur d'inclure la bibliothèque Winsock lors de la compilation

#define TIMEOUT_SEC 0.5   // Durée maximale (en secondes) d'attente pour une connexion à un port

// ==========================================
// Programme : Scanner de ports TCP sur Windows
// Objectif : Tester si les ports 1 à 1024 d'une machine sont ouverts ou fermés
// Cette version : Affiche chaque port, compte les ports ouverts, utilise un timeout rapide
// ==========================================

int main(int argc, char *argv[]) {

    // ==========================================
    // 1. Vérifier si l'utilisateur a fourni une IP
    // ==========================================
    // argc = nombre d'arguments passés au programme (ex: ./scan_ports.exe 127.0.0.1)
    // argv = tableau contenant les arguments
    // argv[0] = nom du programme
    // argv[1] = IP cible
    if (argc != 2) {
        printf("Usage: %s <IP cible>\n", argv[0]);
        printf("Exemple: %s 127.0.0.1\n", argv[0]);
        return 1; // Quitte le programme si pas d'IP donnée
    }

    char *target_ip = argv[1]; // Stocke l'IP cible dans une variable pour faciliter l'utilisation

    // ==========================================
    // 2. Initialiser Winsock (obligatoire sur Windows)
```

```
// ==========================================
WSADATA wsa; // Structure qui stocke les informations de l'API Winsock
if (WSAStartup(MAKEWORD(2,2), &wsa) != 0) {
    // Si WSAStartup échoue, on ne peut pas utiliser le réseau
    printf("Erreur : impossible d'initialiser Winsock\n");
    return 1;
}
// À ce stade, Winsock est prêt à être utilisé pour créer des sockets

printf("Scan des ports TCP sur %s...\n", target_ip);

int open_ports = 0; // Compteur pour savoir combien de ports sont ouverts

// ==========================================
// 3. Boucle pour tester chaque port
// ==========================================
// On teste les ports standards de 1 à 1024 (HTTP, SSH, FTP, SMTP, etc.)
for (int port = 1; port <= 1024; port++) {

    // ==========================================
    // 4. Créer une socket TCP
    // ==========================================
    // Une socket = point de communication pour envoyer/recevoir des données
    // AF_INET = on utilise IPv4 (ex: 192.168.1.1)
    // SOCK_STREAM = connexion TCP (fiable, orientée connexion)
    // IPPROTO_TCP = protocole TCP
    SOCKET sock = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);

    if (sock == INVALID_SOCKET) {
        printf("Erreur de création de socket sur le port %d\n", port);
        continue; // Passe au port suivant si création de socket échoue
    }

    // ==========================================
    // 5. Préparer l'adresse de la machine cible
    // ==========================================
    struct sockaddr_in server_addr; // Structure qui contient l'adresse IP et le port
    server_addr.sin_family = AF_INET; // On précise IPv4
    server_addr.sin_port = htons(port); // Conversion du numéro de port en format réseau
    // htons = Host TO Network Short -> convertit le port du format machine au format réseau

    // inet_pton = Convertit une IP en texte (ex: "127.0.0.1") en format binaire utilisable
    if (inet_pton(AF_INET, target_ip, &server_addr.sin_addr) <= 0) {
        printf("IP invalide : %s\n", target_ip);
        closesocket(sock); // On ferme la socket pour libérer les ressources
        continue; // Passe au port suivant
    }

    memset(server_addr.sin_zero, 0, sizeof(server_addr.sin_zero));
```

```
        // On met à zéro le reste de la structure pour éviter des valeurs indéfinies (bonne
pratique)

        // ==========================================
        // 6. Définir un délai maximum (timeout)
        // ==========================================
        // Timeout = combien de temps attendre avant de considérer un port comme fermé
        DWORD timeout = (DWORD)(TIMEOUT_SEC * 1000); // Conversion en millisecondes (0.5 sec = 500
ms)
        setsockopt(sock, SOL_SOCKET, SO_RCVTIMEO, (const char*)&timeout, sizeof(timeout)); //
Timeout réception
        setsockopt(sock, SOL_SOCKET, SO_SNDTIMEO, (const char*)&timeout, sizeof(timeout)); //
Timeout envoi

        // ==========================================
        // 7. Essayer de se connecter au port
        // ==========================================
        // connect() tente d'ouvrir une connexion TCP avec le port cible
        // Si connect() retourne 0, la connexion a réussi -> port ouvert
        // Sinon -> port fermé ou filtré par un firewall
        if (connect(sock, (struct sockaddr*)&server_addr, sizeof(server_addr)) == 0) {
            printf("Port %d ouvert\n", port); // Affiche que le port est ouvert
            open_ports++; // On incrémente le compteur
        } else {
            printf("Port %d ferme\n", port); // Affiche que le port est fermé
        }

        // ==========================================
        // 8. Fermer la socket après chaque test
        // ==========================================
        closesocket(sock); // Libère les ressources, très important
    }

    // ==========================================
    // 9. Nettoyer Winsock à la fin du programme
    // ==========================================
    WSACleanup(); // Toujours nettoyer Winsock pour libérer la mémoire utilisée

    // ==========================================
    // 10. Afficher un résumé du scan
    // ==========================================
    printf("\nScan terminé.\n");
    printf("Nombre de ports ouverts : %d sur 1024\n", open_ports);
    printf("Tous les ports non listés sont fermés ou filtrés.\n");

    return 0; // Fin du programme
}
```



Créer un programme qui simule une négociation de connexion TCP

— Faire cela dans un langage de bas niveau © :

SERVER :

```c
#include <stdio.h>      // Pour printf() et perror() -> afficher du texte et messages d'erreur
#include <string.h>     // Pour memset() -> mettre des zones mémoire à zéro
#include <winsock2.h>   // Pour les sockets sur Windows
#include <ws2tcpip.h>   // Pour des fonctions réseau supplémentaires (ex: inet_pton)

#pragma comment(lib, "ws2_32.lib") // Indique au compilateur de lier la librairie Winsock

#define PORT 12345 // Port sur lequel le serveur va écouter

int main() {

    // ========================================
    // 1. Initialisation de Winsock
    // ========================================
    // Avant d'utiliser les sockets sur Windows, il faut toujours initialiser l'API réseau Winsock
    WSADATA wsa;
    if (WSAStartup(MAKEWORD(2,2), &wsa) != 0) { // MAKEWORD(2,2) = version 2.2 de Winsock
        printf("Erreur d'initialisation de Winsock\n");
        return 1; // On quitte si impossible d'initialiser
    }
```

```
    // ======================================
    // 2. Création de la socket serveur
    // ======================================
    // SOCK_STREAM = TCP (connexion fiable)
    // AF_INET = IPv4
    // IPPROTO_TCP = protocole TCP
    SOCKET sock = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);

    if (sock == INVALID_SOCKET) { // Vérifie si la socket a été créée correctement
        printf("Erreur de création de socket\n");
        WSACleanup();
        return 1;
    }

    // ======================================
    // 3. Préparer l'adresse du serveur
    // ======================================
    struct sockaddr_in server_addr;
    server_addr.sin_family = AF_INET; // IPv4
    server_addr.sin_port = htons(PORT); // Convertit le numéro de port pour le réseau
    server_addr.sin_addr.s_addr = INADDR_ANY; // Écoute sur toutes les interfaces réseau
    memset(server_addr.sin_zero, 0, sizeof(server_addr.sin_zero)); // Remplit le reste de la
structure avec des zéros

    // ======================================
    // 4. Lier la socket à l'adresse
    // ======================================
    if (bind(sock, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
        printf("Erreur de bind\n");
        closesocket(sock);
        WSACleanup();
        return 1;
    }

    // ======================================
    // 5. Mettre la socket en mode écoute
    // ======================================
    // Le serveur va attendre des connexions entrantes
    if (listen(sock, 1) < 0) { // 1 = nombre max de clients en file d'attente
        printf("Erreur de listen\n");
        closesocket(sock);
        WSACleanup();
        return 1;
    }

    printf("Serveur prêt sur le port %d\n", PORT);

    // ======================================
```

```c
// 6. Attente d'un client
// ======================================
SOCKET client_sock;
struct sockaddr_in client_addr;
int addr_len = sizeof(client_addr);

client_sock = accept(sock, (struct sockaddr*)&client_addr, &addr_len);
// accept() bloque jusqu'à ce qu'un client se connecte
if (client_sock == INVALID_SOCKET) {
    printf("Erreur d'accept\n");
    closesocket(sock);
    WSACleanup();
    return 1;
}

printf("Client connecté !\n");

// ======================================
// 7. Simulation handshake TCP
// ======================================
// Le vrai handshake TCP est SYN → SYN-ACK → ACK
// Ici on simule juste les messages pour l'apprentissage
char buffer[1024];

// Attendre SYN du client
recv(client_sock, buffer, sizeof(buffer), 0);
printf("Serveur: reçu SYN du client\n");

// Envoyer SYN-ACK
send(client_sock, "SYN-ACK", 8, 0);
printf("Serveur: envoyé SYN-ACK\n");

// Attendre ACK
recv(client_sock, buffer, sizeof(buffer), 0);
printf("Serveur: reçu ACK, connexion établie !\n");

// ======================================
// 8. Fermeture des sockets
// ======================================
closesocket(client_sock);
closesocket(sock);

// ======================================
// 9. Nettoyage Winsock
// ======================================
WSACleanup();

printf("Serveur terminé.\n");
return 0;
```

```
}
```

Client :

```c
#include <stdio.h>
#include <string.h>
#include <winsock2.h>
#include <ws2tcpip.h>

#pragma comment(lib, "ws2_32.lib") // Lien avec Winsock

#define PORT 12345 // Port sur lequel le serveur écoute

int main() {

    // ======================================
    // 1. Initialisation de Winsock
    // ======================================
    WSADATA wsa;
    if (WSAStartup(MAKEWORD(2,2), &wsa) != 0) {
        printf("Erreur d'initialisation de Winsock\n");
        return 1;
    }

    // ======================================
    // 2. Création de la socket client
    // ======================================
    SOCKET sock = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (sock == INVALID_SOCKET) {
        printf("Erreur de création de socket\n");
        WSACleanup();
        return 1;
    }

    // ======================================
    // 3. Préparer l'adresse du serveur
    // ======================================
    struct sockaddr_in server_addr;
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT); // Convertir le port pour le réseau
    inet_pton(AF_INET, "127.0.0.1", &server_addr.sin_addr); // IP locale
    memset(server_addr.sin_zero, 0, sizeof(server_addr.sin_zero));

    // ======================================
    // 4. Connexion au serveur
    // ======================================
    if (connect(sock, (struct sockaddr*)&server_addr, sizeof(server_addr)) < 0) {
        printf("Erreur de connexion\n");
```

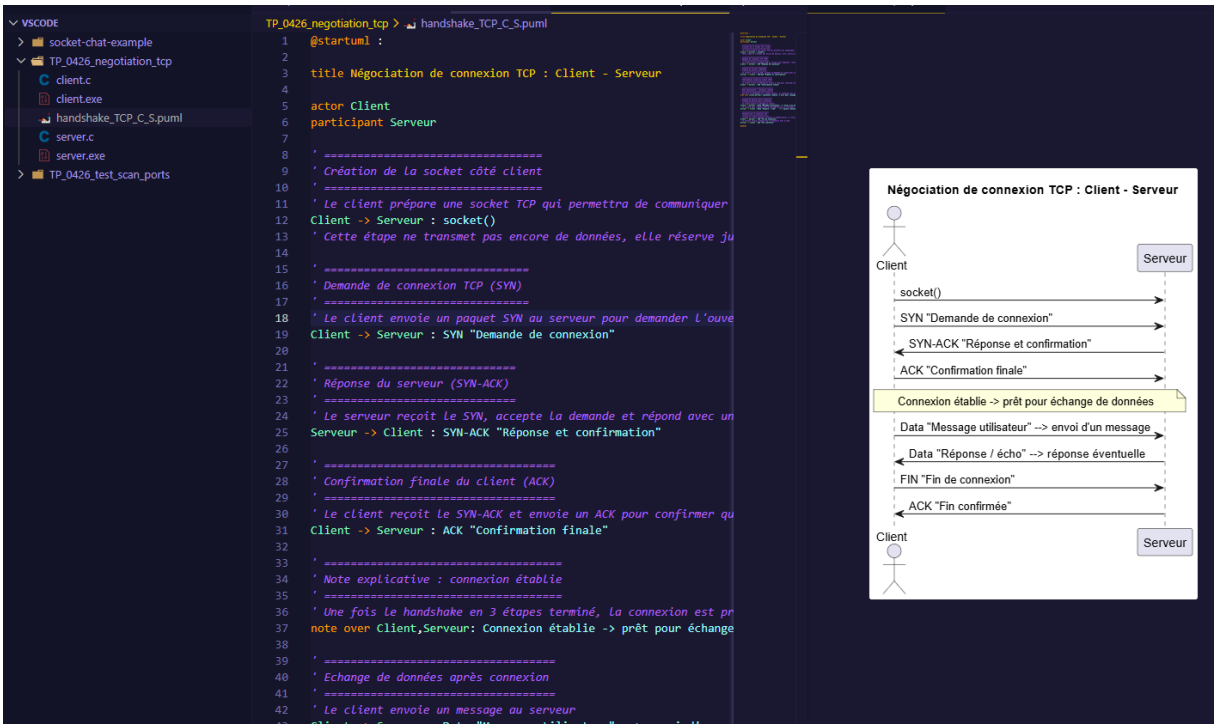
```
        closesocket(sock);
        WSACleanup();
        return 1;
    }

    printf("Connecté au serveur !\n");

    // ======================================
    // 5. Simulation handshake TCP
    // ======================================
    char buffer[1024];

    // Envoyer SYN
    send(sock, "SYN", 4, 0);
    printf("Client: envoyé SYN\n");

    // Attendre SYN-ACK
    recv(sock, buffer, sizeof(buffer), 0);
    printf("Client: reçu %s\n", buffer);

    // Envoyer ACK
    send(sock, "ACK", 4, 0);
    printf("Client: envoyé ACK, connexion établie !\n");

    // ======================================
    // 6. Fermeture de la socket et nettoyage
    // ======================================
    closesocket(sock);
    WSACleanup();

    printf("Client terminé.\n");
    return 0;
}
```

— Faire le diagramme de séquence associé (UML)

Dans plantuml – vscode directement :

```
@startuml :

title Négociation de connexion TCP : Client - Serveur

actor Client
participant Serveur

' ================================
' Création de la socket côté client
' ================================
```

```
' Le client prépare une socket TCP qui permettra de communiquer avec le serveur
Client -> Serveur : socket()
' Cette étape ne transmet pas encore de données, elle réserve juste le canal de communication

' ==============================
' Demande de connexion TCP (SYN)
' ==============================
' Le client envoie un paquet SYN au serveur pour demander l'ouverture d'une connexion
Client -> Serveur : SYN "Demande de connexion"

' ============================
' Réponse du serveur (SYN-ACK)
' ============================
' Le serveur reçoit le SYN, accepte la demande et répond avec un SYN-ACK pour confirmer
Serveur -> Client : SYN-ACK "Réponse et confirmation"

' ===================================
' Confirmation finale du client (ACK)
' ===================================
' Le client reçoit le SYN-ACK et envoie un ACK pour confirmer que la connexion est établie
Client -> Serveur : ACK "Confirmation finale"

' ====================================
' Note explicative : connexion établie
' ====================================
' Une fois le handshake en 3 étapes terminé, la connexion est prête pour échanger des données
note over Client,Serveur: Connexion établie -> prêt pour échange de données

' ===================================
' Echange de données après connexion
' ===================================
' Le client envoie un message au serveur
Client -> Serveur : Data "Message utilisateur" --> envoi d'un message
' Le serveur peut répondre avec un message d'écho ou traitement
Serveur -> Client : Data "Réponse / écho"    --> réponse éventuelle

' ==============================
' Fermeture de la connexion TCP
' ==============================
' Lorsque le client souhaite terminer la communication, il envoie un paquet FIN
Client -> Serveur : FIN "Fin de connexion"
' Le serveur accuse réception de la fermeture avec un ACK
Serveur -> Client : ACK "Fin confirmée"

@enduml
```



Créer un programme qui simule une négociation de connexion TLS

— Faire cela dans un langage de bas niveau © :

SERVER :

```
#include <stdio.h>      // Pour printf() et perror() -> afficher des messages
#include <stdlib.h>     // Pour exit() et fonctions utilitaires
#include <string.h>     // Pour memset() -> mettre une zone mémoire à zéro
#include <winsock2.h>   // Pour utiliser les sockets sur Windows
#include <ws2tcpip.h>   // Pour inet_pton et fonctions réseau avancées

#pragma comment(lib, "ws2_32.lib") // Permet de lier automatiquement la librairie Winsock

#define PORT 4433          // Port simulé pour TLS (HTTPS par défaut c'est 443)
#define BUFFER_SIZE 1024   // Taille du buffer pour les messages

int main() {
    WSADATA wsa;                // Structure pour initialiser Winsock
    SOCKET server_socket;       // Socket serveur
    SOCKET client_socket;       // Socket pour le client connecté
    struct sockaddr_in server_addr, client_addr; // Structures pour stocker les adresses réseau
    int client_len = sizeof(client_addr);       // Taille de la structure client
    char buffer[BUFFER_SIZE];  // Buffer pour envoyer et recevoir des données

    // ==========================================
    // Initialiser Winsock
    // ==========================================
```

```c
printf("Initialisation de Winsock...\n");
if (WSAStartup(MAKEWORD(2,2), &wsa) != 0) { // Version 2.2 de Winsock
    printf("Erreur d'initialisation de Winsock\n");
    return 1; // Quitte si erreur
}

// ==========================================
// Créer la socket TCP du serveur
// ==========================================
server_socket = socket(AF_INET, SOCK_STREAM, 0);
if (server_socket == INVALID_SOCKET) {
    printf("Erreur création socket\n");
    WSACleanup(); // Nettoyage de Winsock
    return 1;
}

// ==========================================
// Configurer l'adresse du serveur
// ==========================================
server_addr.sin_family = AF_INET;         // IPv4
server_addr.sin_addr.s_addr = INADDR_ANY; // Accepte toutes les interfaces réseau locales
server_addr.sin_port = htons(PORT);       // Convertit le port en format réseau (big-endian)

// ==========================================
// Lier la socket à l'adresse et au port
// ==========================================
if (bind(server_socket, (struct sockaddr*)&server_addr, sizeof(server_addr)) == SOCKET_ERROR) {
    printf("Erreur lors du bind\n");
    closesocket(server_socket);
    WSACleanup();
    return 1;
}

// ==========================================
// Mettre le serveur en écoute
// ==========================================
listen(server_socket, 1); // 1 = taille de la file d'attente
printf("Serveur TLS simulé en écoute sur le port %d...\n", PORT);

// ==========================================
// Attendre et accepter la connexion d'un client
// ==========================================
client_socket = accept(server_socket, (struct sockaddr*)&client_addr, &client_len);
if (client_socket == INVALID_SOCKET) {
    printf("Erreur lors de l'acceptation de connexion\n");
    closesocket(server_socket);
    WSACleanup();
    return 1;
}
```

```c
    printf("Client connecté !\n");

    // ==========================================
    // 1. Recevoir ClientHello
    // ==========================================
    memset(buffer, 0, BUFFER_SIZE);           // Mettre le buffer à zéro
    recv(client_socket, buffer, BUFFER_SIZE, 0); // Lire les données envoyées par le client
    printf("Reçu du client : %s\n", buffer); // Affiche le ClientHello simulé

    // ==========================================
    // 2. Envoyer ServerHello + certificat
    // ==========================================
    strcpy(buffer, "ServerHello: TLS 1.2, AES256, random1234"); // Message simulé
    send(client_socket, buffer, strlen(buffer), 0);            // Envoyer au client

    strcpy(buffer, "Certificate: CertificatServeurSimulé");    // Certificat simulé
    send(client_socket, buffer, strlen(buffer), 0);

    // ==========================================
    // 3. Recevoir ClientKeyExchange (clé pré-master)
    // ==========================================
    memset(buffer, 0, BUFFER_SIZE);           // Réinitialiser le buffer
    recv(client_socket, buffer, BUFFER_SIZE, 0);
    printf("Clé pré-master reçue : %s\n", buffer); // Affiche la clé pré-master

    // ==========================================
    // 4. Passage en mode chiffré
    // ==========================================
    strcpy(buffer, "ChangeCipherSpec: passage en mode chiffré"); // Simulation
    send(client_socket, buffer, strlen(buffer), 0);

    // ==========================================
    // 5. Handshake terminé
    // ==========================================
    strcpy(buffer, "Finished: handshake TLS terminé côté serveur"); // Indique que TLS est prêt
    send(client_socket, buffer, strlen(buffer), 0);

    // ==========================================
    // 6. Échange de données simulé
    // ==========================================
    while(1) {
        memset(buffer, 0, BUFFER_SIZE);               // Nettoyer le buffer
        int bytes = recv(client_socket, buffer, BUFFER_SIZE, 0); // Lire le message
        if (bytes <= 0) break;                        // Si rien reçu => client déconnecté

        printf("Message chiffré reçu : %s\n", buffer); // Affiche le message simulé

        // Réponse du serveur
        strcpy(buffer, "Réponse serveur chiffrée");
```

```c
        send(client_socket, buffer, strlen(buffer), 0);
    }

    printf("Client déconnecté, fermeture du serveur...\n");

    closesocket(client_socket); // Fermer la socket client
    closesocket(server_socket); // Fermer la socket serveur
    WSACleanup();               // Nettoyage final de Winsock

    return 0;
}
```

CLIENT :

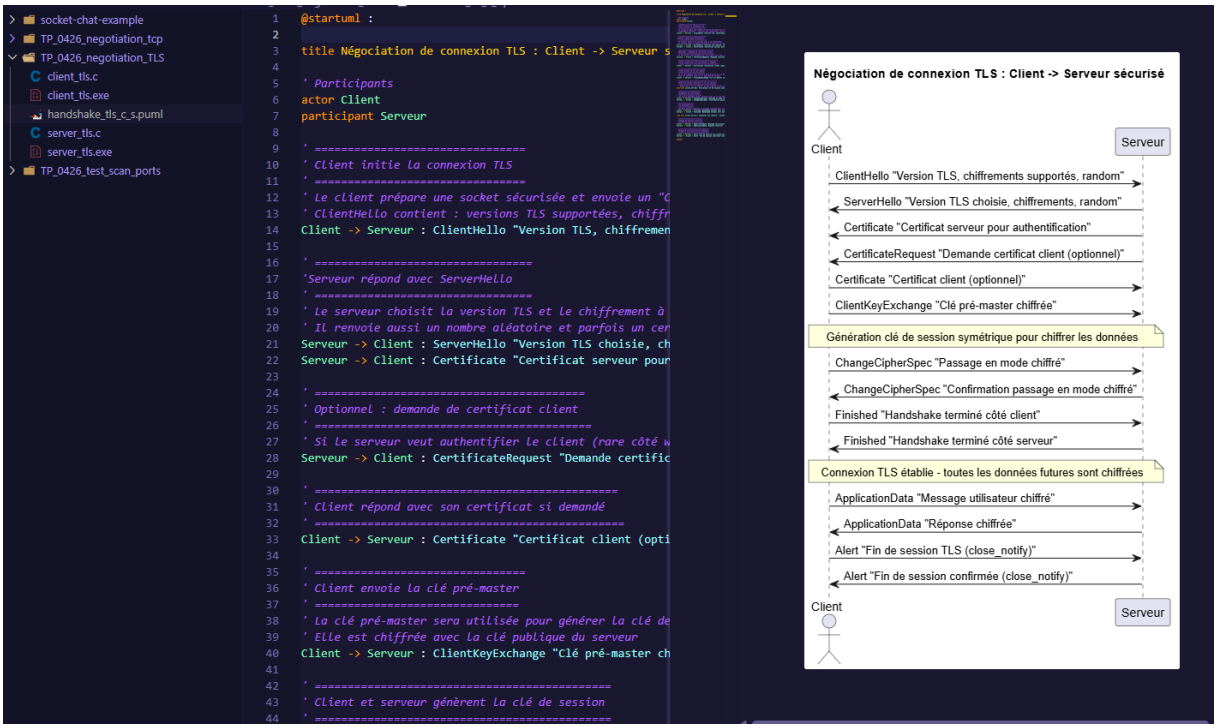
```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <winsock2.h>
#include <ws2tcpip.h>

#pragma comment(lib, "ws2_32.lib") // Lier Winsock automatiquement

#define SERVER_IP "127.0.0.1" // IP du serveur local
#define PORT 4433             // Port du serveur TLS simulé
#define BUFFER_SIZE 1024      // Taille du buffer pour envoyer/recevoir

int main() {
    WSADATA wsa;
    SOCKET sock;
    struct sockaddr_in server_addr;
    char buffer[BUFFER_SIZE];

    // ==========================================
    // Initialiser Winsock
    // ==========================================
    printf("Initialisation de Winsock...\n");
    if (WSAStartup(MAKEWORD(2,2), &wsa) != 0) {
        printf("Erreur d'initialisation Winsock\n");
        return 1;
    }

    // ==========================================
    // Créer la socket TCP pour le client
    // ==========================================
    sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock == INVALID_SOCKET) {
        printf("Erreur création socket\n");
        WSACleanup();
```

```
    return 1;
}

// ==========================================
// Configurer l'adresse du serveur
// ==========================================
server_addr.sin_family = AF_INET; // IPv4
server_addr.sin_port = htons(PORT);
inet_pton(AF_INET, SERVER_IP, &server_addr.sin_addr); // Convertir l'IP en binaire

// ==========================================
// Connexion au serveur
// ==========================================
if (connect(sock, (struct sockaddr*)&server_addr, sizeof(server_addr)) != 0) {
    printf("Impossible de se connecter au serveur\n");
    closesocket(sock);
    WSACleanup();
    return 1;
}

printf("Connecté au serveur !\n");

// ==========================================
// 1. Envoyer ClientHello
// ==========================================
strcpy(buffer, "ClientHello: TLS 1.2, AES256, random5678"); // Simulé
send(sock, buffer, strlen(buffer), 0);

// ==========================================
// 2. Recevoir ServerHello + certificat
// ==========================================
memset(buffer, 0, BUFFER_SIZE);
recv(sock, buffer, BUFFER_SIZE, 0);
printf("Reçu du serveur : %s\n", buffer); // ServerHello

memset(buffer, 0, BUFFER_SIZE);
recv(sock, buffer, BUFFER_SIZE, 0);
printf("Reçu du serveur : %s\n", buffer); // Certificat

// ==========================================
// 3. Envoyer ClientKeyExchange (clé pré-master)
// ==========================================
strcpy(buffer, "ClientKeyExchange: clé pré-master simulée");
send(sock, buffer, strlen(buffer), 0);

// ==========================================
// 4. Passage en mode chiffré
// ==========================================
memset(buffer, 0, BUFFER_SIZE);
```

```
    recv(sock, buffer, BUFFER_SIZE, 0);
    printf("%s\n", buffer); // ChangeCipherSpec

    // ========================================
    // 5. Handshake terminé
    // ========================================
    memset(buffer, 0, BUFFER_SIZE);
    recv(sock, buffer, BUFFER_SIZE, 0);
    printf("%s\n", buffer); // Finished

    // ========================================
    // 6. Échange de messages simulé
    // ========================================
    while(1) {
        printf("Entrez un message (ou 'exit' pour quitter) : ");
        fgets(buffer, BUFFER_SIZE, stdin);                // Lire l'entrée utilisateur
        buffer[strcspn(buffer, "\n")] = 0;               // Supprimer le retour chariot

        if (strcmp(buffer, "exit") == 0) break;          // Quitter si l'utilisateur tape exit

        send(sock, buffer, strlen(buffer), 0);           // Envoyer le message au serveur

        memset(buffer, 0, BUFFER_SIZE);
        recv(sock, buffer, BUFFER_SIZE, 0);             // Recevoir la réponse du serveur
        printf("Réponse serveur : %s\n", buffer);
    }

    printf("Déconnexion du serveur...\n");
    closesocket(sock); // Fermer la socket client
    WSACleanup();      // Nettoyage final de Winsock

    return 0;
}
```

— Faire le diagramme de séquence associé (UML)

Dans plantuml – vscode directement :



Mon Git :

https://github.com/Askhany/TP-Guillemette-Avril